

Tensor-network-based PDE solvers

Author: Pau Torrente Badia

Facultat de Física, Universitat de Barcelona, Diagonal 645, 08028 Barcelona, Spain.

Supervisor: Stefano Carignano

Abstract: In this work we overview Tensor-Network based PDE solvers, focusing on understanding the basic building blocks that allow integrating differential operators and grid-based methods within the framework. We start off from the Tensor Train Cross formalism in order to encode arbitrary functions and then we study how the prolongation and optimization procedures presented in [1] can be applied to arbitrary functions in the context of the Poisson equation.

I. INTRODUCTION

Partial differential equations lie at the core of many, if not all, physical and engineering theories. With this, there is a great interest in new methods to increase efficiency and/or speed, especially for large scale problems, where memory and convergence problems arise. To deal with these issues, Tensor Networks have emerged as a very promising alternative, due to their efficiency at compressing information in a very wide range of applications and use cases. In this work, we aim to introduce and evaluate the framework for tackling PDE's presented in [1], using the Poisson equation as a benchmark. In order to add flexibility to the numerical pipeline and be able to test arbitrary cases, we also combine the optimization procedures used to solve the actual PDE's, with the Tensor Train Cross (TT-Cross) interpolating procedure in the quantized TT (QTT) format presented in [2] and previously studied by the authors of this work in [3].

This brief report, then, is organized as follows. In Sec. II, the formalism of tensor networks is briefly presented, alongside a graphical introduction to the TT-Cross workflow in the QTT format. In Sec. III the optimization pipeline from [1] is showcased. In IV, the Poisson equation is solved for a simple case with a polynomial source term. Lastly, the conclusions of the project are discussed in Sec. V.

II. THE TT-CROSS APPROXIMATION IN THE QTT FORMAT

A. Binarization of single variable function

Let $f : D \subset \mathbb{R} \rightarrow \mathbb{C}$ be a complex valued scalar function. Our goal is to encode its information in a compressed way in a tensor network to be able to manipulate it further. In order to do so, we first discretize the domain D :

$$D \rightarrow X = \{x_i\} \subset D ; i = 1, \dots, n, \quad (1)$$

and then we introduce the following change of variables that, effectively, "binarizes" the index i :

$$i \leftrightarrow (i_1, \dots, i_d), \quad i = \sum_{p=1}^d i_p 2^{p-1}, \quad i_p = 0, 1. \quad (2)$$

This results in turning out our studied function f into a d -dimensional function over a discrete grid, covering a total of 2^d points.

B. Function encoding onto a Tensor Network

Once we have transformed our function into being defined over a d -dimensional grid, we can propose a rank- d tensor that contains all the entries of the function:

$$A(i_1, \dots, i_d) = f(x_{i_1}, \dots, x_{i_d}) \\ i_s = 0, 1; s = 1, \dots, d \quad (3)$$

The number of entries of this tensor, however, scales exponentially with the chosen fineness of the grid, given by d , but it can be approximated via a tensor network of the form depicted in Fig. 1 that represents the construction given by the TT-Cross approximation [4]:

$$A(i_1, \dots, i_N) \approx \tilde{A}(i_1, \dots, i_N) = A(i_1, \mathcal{J}^{(1)}) \times \\ \times \left[A(\mathcal{I}^{(1)}, \mathcal{J}^{(1)}) \right]^{-1} A(\mathcal{I}^{(1)}, i_2, \mathcal{J}^{(2)}) \dots A(\mathcal{I}^{(N-1)}, i_N), \quad (4)$$

where $\mathcal{I}^{(k)}$ and $\mathcal{J}^{(k)}$ are sets of multi-indices $\{i_1, \dots, i_k\}$ and $\{i_{k+1}, \dots, i_d\}$, respectively. The important thing about this construction is that it can be built from the ground up by sampling the studied function in the discretized grid. With this, the information required is very small compared to the exponential number of points needed to build the full tensor. A comprehensive introduction to tensor networks and their compression properties can be found in [5] and a much more detailed description of the TT-Cross approximation as well as these algorithms to build it can be found in [3, 4, 6]. For consistency purposes in the following steps, the output tensor trains of the TT-Cross procedure shown in Fig. 1 will have their $B^{(k)}$ tensor contracted into the neighboring $A^{(k)}$ ones.

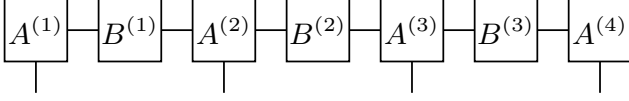


FIG. 1: Tensor diagram representing the TT-Cross decomposition given in Eq (4) for a simple case with $N = 4$. In the figure, the 3-legged tensors labeled as $A^{(k)}$ refer to $A(\mathcal{I}^{(k-1)}, i_k, \mathcal{J}^{(k+1)})$, the 2-legged ones labeled as $B^{(k)}$ refer to $[A(\mathcal{I}^{(k)}, \mathcal{J}^{(k)})]^{-1}$.

III. THE MULTIGRID RENORMALIZATION PROCEDURE

Now that we have a way to encode functions in networks of tensors, we can start tackling equations of the form:

$$\mathcal{L}f = g \quad (5)$$

Where $\mathcal{L}$ is our 1D differential operator of choice, g what we will call target function and f the function that we want to find satisfying the differential equation. In the context of finite difference methods, Eq. (5) becomes a system of linear equations which can be solved with regular linear algebra solvers, but that for very large instances, are known to face convergence and memory issues. In order to tackle these two problems, the Multigrid Renormalization procedure from [1] presents an iterative process that combines an optimization step, where the system of linear equations is solved as an optimization procedure on top of a tensor train ansatz, and a consequent prolongation procedure that refines the grid by a factor 2 by interpolation. With this, starting from a very coarse grid, consecutive steps of this kind refine and optimize the function iteratively on top of the tensor network ansatz, facilitating convergence and also avoiding memory issues.

A. Prolongation

The first part of each algorithm step is to refine our grid by a factor 2, such that the new tensor train encodes the entries of the function interpolating the points given by the previous construction. This is done by the tensor train operator $\mathcal{P}$ depicted in Fig. 2, whose entries are given in Ref. [1]. Since the full contraction of the operator onto the network encoding our function would increase the size of the contracted indices exponentially [7], an approximate approach is used, which aims to reduce the cost function given by:

$$c(|f\rangle) = \|\mathcal{P}|c\rangle - |f\rangle\|^2, \quad (6)$$

where $|c\rangle$ and $|f\rangle$ represent, respectively, the networks encoding the coarse and fine descriptions of the function. This optimization is performed in a two-site sweeping

procedure following [7], that directly updates the tensors without the need of any third-party solver granted by the orthogonal form of the network at each step.

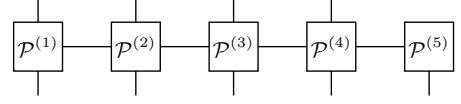


FIG. 2: Tensor network representing the Prolongation operator $\mathcal{P}$ for the case $N = 5$.

B. PDE optimization

The second step in the optimization procedure is the one that involves solving the actual PDE, this is, optimizing our function representation of the solution of our equation. This is done with a very similar procedure to that of the prolongation step, sweeping along the tensor train and optimizing our tensors according to either one of the following loss functions:

$$\begin{cases} c_1(|f\rangle) = \|\mathcal{L}|f\rangle - |g\rangle\|^2 \\ c_2(|f\rangle) = \langle f|\mathcal{L}|f\rangle - \langle f|g \rangle \end{cases} \quad (7)$$

where $|f\rangle$ is the current best solution obtained in the prolongation procedure. Notice that in this setting the second cost function does not have a factor 2 multiplying the term $\langle f|g\rangle$ as in the original reference, since in our case we are treating $\langle f|$ and $|f\rangle$ as two different objects, such that the derivative results in:

$$\frac{\partial c_2}{\partial \langle f|} = \mathcal{L}|f\rangle - |g\rangle \quad (8)$$

With this, therefore, minimizing the cost function to reach the minimum that results in Eq. (8) being null, gives us the solution of our PDE.

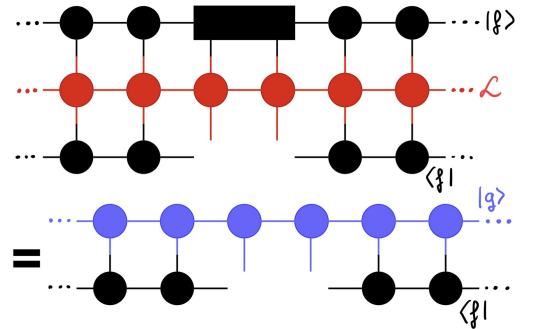


FIG. 3: Diagrammatic notation of the system of linear equations that results from imposing $\frac{\partial c}{\partial \langle f|}_{\text{loc}} = 0$, where the "loc" subscript refers to the minimization over only the local tensors. The upper contraction represents the product of a linear operator A times an unknown solution x , given by the black square. The lower contraction represents the non zero term b , resulting in the linear system $Ax = b$.

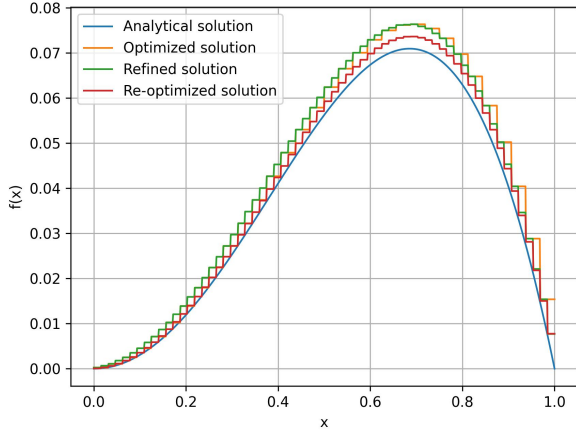


FIG. 4: Comparison of the function values retrieved from the tensor trains in consecutive optimization-prolongation-optimization steps alongside the analytical solution given by Eq. (11). The process starts from a tensor train of length 5 filled with random entries.

It is very important to note that in contrast to what happened in the prolongation step, now the operator acts on the optimized function, not the target one. This makes the optimization non-trivial, without a direct local update, and requires the usage of a linear equation solver at each update, which for simplicity purposes, the *gmres* solver from the *SciPy* python library [8] has been the one chosen. The local linear problem to be solved for a tensor outside the edges of the network in order to minimize the cost function c_2 from Eq. (7) is depicted in Fig. 3 as an example.

IV. TACKLING THE POISSON EQUATION

To test the two procedures described in Sec. III we will use the 1D-Poisson equation in the interval $x = [0, 1]$

with Dirichlet boundary conditions as a benchmark:

$$\nabla^2 f = g, \quad (9)$$

where the MPO representation of the Laplace operator in QTT format already encoding the boundary conditions desired can be found in Ref. [1]. As a source term $|g\rangle$, we will proceed with a TT-Cross interpolation of the following polynomial:

$$g(x) = 0.6 + 0.6x - 7.2x^2 + 4x^3, \quad (10)$$

in order to easily cross check the results with the true solution of the PDE:

$$f^{\text{true}}(x) = 0.3x^2 + 0.1x^3 - 0.6x^4 + 0.2x^5 \quad (11)$$

Since polynomials are relatively simple functions, we proceed with the PDE optimization first on a coarse grid initializing the algorithm with a random tensor train. Then, we apply the prolongation procedure to obtain a refined description of the solution that is optimized again. The results from this three steps are shown in Fig. 4, where we can see how the algorithm is able to infer the structure of the function and how the iterative process brings the tensor train description closer to the true solution of the PDE, even with just one setp of the process.

V. CONCLUSIONS

In this work we have reviewed the iterative Multigrid Renormalization procedure as a tool for solving PDE's. We have combined it also with TT-Cross interpolation techniques in order to tackle arbitrary problems in 1D. By studying a simple polynomial as a benchmark, we have been able to check that the process can tackle arbitrary functions and that the multi-step procedure is able to capture the structure of the functions and converge onto the desired solution.

-
- [1] M. Lubasch, P. Moinier, and D. Jaksch, “Multigrid renormalization,” *Journal of Computational Physics*, vol. 372, pp. 587–602, 2018.
 - [2] D. Savostyanov and I. Oseledets, “Fast adaptive interpolation of multi-dimensional arrays in tensor train format,” in *The 2011 International Workshop on Multidimensional (nD) Systems*, pp. 1–8, 2011.
 - [3] P. Torrente, “Tensor network based integration methods,” bachelor’s thesis, Universitat de Barcelona, Facultat de Física, 2024.
 - [4] I. Oseledets and E. Tyrtyshnikov, “Tt-cross approximation for multidimensional arrays,” *Linear Algebra and its Applications*, vol. 432, no. 1, pp. 70–88, 2010.
 - [5] R. Orús, “A practical introduction to tensor networks: Matrix product states and projected entangled pair states,” *Annals of Physics*, vol. 349, p. 117–158, Oct. 2014.
 - [6] S. Dolgov and D. Savostyanov, “Parallel cross interpolation for high-precision calculation of high-dimensional integrals,” *Computer Physics Communications*, vol. 246, p. 106869, 2020.
 - [7] P. García-Molina, L. Tagliacozzo, and J. J. García-Ripoll, “Global optimization of mps in quantum-inspired numerical analysis,” 2024.
 - [8] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, and SciPy 1.0 Contributors, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.